# TAREA GLOBAL: PROGRAMACIÓN ESTRUCTURADA EN PYTHON (UT2)

## 1. FICHA TÉCNICA DE LA ACTIVIDAD

| Parámetro | Detalle |
|---|---|
| **Módulo** | Programación con Python (Optativa - ASIR) |
| **Unidad** | UT2. Programación Estructurada |
| **Duración** | 13 horas estimadas |
| **Herramientas** | Visual Studio Code, Python 3, GNU/Linux |
| **Formato** | Archivo .zip con scripts .py y memoria técnica en PDF |

## 2. DESCRIPCIÓN GENERAL Y OBJETIVOS

Esta tarea constituye el núcleo práctico de la unidad de Programación Estructurada. El objetivo es asentar las bases sintácticas del lenguaje Python, evolucionando desde estructuras secuenciales básicas hasta la implementación de lógica compleja mediante bifurcaciones condicionales, bucles de control y modularización por funciones, siempre bajo una óptica de **Administración de Sistemas y Redes (ASIR)**.

## 3. CUADERNO DE ACTIVIDADES

### GRUPO A: EJERCICIOS INICIALES (Asimilación, Secuencialidad y Funciones)

#### A.1. Secuenciales (Entrada, Proceso, Salida)

1. **Cálculo Geométrico:** Desarrolla un script que solicite la base y la altura de un rectángulo para calcular y mostrar su perímetro y área total.
2. **Aritmética Básica:** Crea un programa que pida dos números reales y realice las operaciones de suma, resta, multiplicación y división, presenting los resultados formateados.

3. **Conversor Térmico:** Escribe un script que reciba una temperatura en grados Fahrenheit y la transforme a su equivalente en grados Celsius aplicando la fórmula C = (F - 32) * 5/9.
4. **Media de Datos:** Diseña un programa que capture tres valores numéricos y devuelva la media aritmética ponderada de los mismos.
5. **Gestión de Tiempo:** Implementa un script que convierta una cantidad bruta de minutos introducida por el usuario en su desglose exacto de horas y minutos restantes.
6. **Cálculo de Nómina:** Programa un calculador de sueldo que sume al salario base un 10% de comisión sobre tres ventas realizadas, mostrando el total a percibir.
7. **Descuento Comercial:** Crea un script que aplique un descuento del 15% sobre el valor total de una compra solicitada por teclado e informe del precio final.
8. **Evaluación Académica:** Desarrolla un algoritmo que calcule la nota final de un alumno basándose en tres calificaciones parciales, un examen final y un trabajo práctico.
9. **Distancia Numérica:** Escribe un programa que determine la distancia absoluta (valor positivo) entre dos números cualesquiera introducidos por el usuario.

### A.2. Condicionales y Validación

10. **Login Seguro:** Simula un sistema de autenticación solicitando usuario y contraseña (definidos previamente en variables). El script debe permitir un máximo de 3 intentos fallidos; si se exceden, debe mostrar un mensaje de "Acceso bloqueado por seguridad" y terminar la ejecución.
11. **Conversor de Moneda:** Crea un menú que permita convertir cantidades entre USD y EUR (tasa fija 0.95). El script debe validar que la cantidad ingresada sea un número positivo. Debe incluir un bucle que permita al usuario realizar múltiples conversiones seguidas y salir solo cuando elija una opción específica.
12. **Cálculo de IVA:** Diseña un programa que calcule el precio final de un producto. Solicita el precio base y el tipo de IVA (21%, 10% o 4%). El script debe validar mediante condicionales que el tipo introducido sea válido; si se introduce un valor distinto, debe mostrar un error específico y terminar o pedir el dato de nuevo.
13. **Rangos de Impares:** Solicita dos números enteros al usuario. Valida mediante condicionales que el primero sea menor que el segundo; si no es así, solicita los datos de

nuevo o muestra un error. Si los datos son correctos, recorre el rango y muestra por pantalla únicamente los números impares presentes.

14. **Filtro de Positivos y Paridad:** Crea un sistema de entrada que solicite números indefinidamente. El programa solo procesará números de 3 o más dígitos (validación). Finalizará cuando se introduzca un número negativo o cero. Para cada número válido procesado, debe indicar si es par o impar.

### A.3. Introducción a Funciones

15. **Saludo Personalizado:** Define una función llamada `saludar(nombre)` que reciba un nombre como parámetro e imprima un mensaje de bienvenida. Llama a la función solicitando previamente el nombre al usuario por teclado.

16. **Calculadora de Área con Función:** Crea una función `calcular_area_triangulo(base, altura)` que devuelva (usando `return`) el área de un triángulo. Solicita los datos en el programa principal e imprime el resultado devuelto.

17. **Comprobador de Paridad:** Diseña una función `es_par(numero)` que devuelva `True` si el número es par y `False` si es impar. Utiliza esta función en el programa principal para comprobar un número introducido por el usuario.

18. **Conversor de Unidades (MB a GB):** Define una función `mb_a_gb(megabytes)` que transforme una cantidad de megabytes en gigabytes y devuelva el resultado. Prueba la función con varios valores.

19. **Número Mayor de Dos:** Implementa la función `obtener_mayor(a, b)` que reciba dos números y retorne el valor del número mayor. Si son iguales, puede devolver cualquiera de los dos.

20. **Cálculo de Porcentaje:** Crea la función `calcular_porcentaje(cantidad, porcentaje)` que reciba un valor numérico y el porcentaje a aplicar, devolviendo la cantidad resultante.

21. **Validador de Longitud de Contraseña:** Diseña una función `es_clave_segura(clave)` que reciba una cadena de texto y devuelva `True` si tiene 8 o más caracteres, o `False` en caso contrario.

22. **Generador de Email Corporativo:** Escribe una función `generar_email(nombre, apellido, dominio)` que reciba el nombre, el apellido y el dominio empresarial (ej.

`"empresa.com"`) y retorne el correo en formato en minúsculas: `nombre.apellido@dominio`.

23. **Tabla de Multiplicar:** Crea una función `mostrar_tabla_multiplicar(numero)` que imprima directamente la tabla de multiplicar del 1 al 10 del número pasado por parámetro (no requiere usar `return`).

24. **Conversor de Segundos a Formato Horas:** Diseña una función `formatear_tiempo(segundos_totales)` que convierta una cantidad entera de segundos en una cadena con el formato `"Xh Ym Zs"` y la devuelva.

## GRUPO B: EJERCICIOS PRÁCTICOS (Lógica, Sistemas y Redes)

### B.1. Procesamiento de Cadenas y Logs

25. **Extracción de IP de Log:** Dado un texto con el formato `"Origen: 192.168.1.50"`, separa el texto para obtener únicamente la dirección IP y muéstrala en pantalla.

    💡 **Ayuda de Python:** Utiliza el método de cadenas `split()` para dividir el texto en partes tomando como separador los espacios.

26. **Cálculo de Duración en Segundos:** Solicita al usuario los minutos y segundos de duración de una tarea (ej. 3 minutos y 40 segundos) y calcula el tiempo total equivalente expresado solo en segundos.

    💡 **Ayuda de Python:** No necesitas funciones avanzadas; puedes convertir los valores introducidos a entero usando `int()` y aplicar la fórmula: `totales = (minutos * 60) + segundos`.

27. **Detector de Error en Log:** Solicita una línea de texto de log e indica si contiene la palabra `"ERROR"` o `"CRITICAL"`.

    💡 **Ayuda de Python:** Puedes usar el operador de pertenencia `in` (ejemplo: `if "ERROR" in texto:`) o convertir el texto a mayúsculas con `upper()` para facilitar la búsqueda.

28. **Estimador de Tiempo de Descarga:** Crea un script que pida el tamaño de un archivo en MB y la velocidad de descarga en MB/s, y muestre cuántos segundos tardará en descargarse.

💡 **Ayuda de Python:** Realiza una división simple de `tamaño / velocidad` y puedes usar la función `round(resultado, 2)` para redondear a dos decimales.

29. **Comprobador de SLA Mensual:** Pida los minutos que un servidor ha estado fuera de servicio durante el mes. Si ha superado los 45 minutos, mostrará el mensaje "SLA Incumplido"; en caso contrario, "SLA Correcto".

    💡 **Ayuda de Python:** Utiliza una estructura condicional básica `if / else` evaluando si la variable ingresada es mayor a 45.

30. **Simulador de Estado de Servicio:** Crea un script que pida el nombre de un servicio (ej. Apache) y un número de estado (0 para detenido, 1 para activo). Debe mostrar un mensaje descriptivo del tipo "El servicio Apache está Activo".

    💡 **Ayuda de Python:** Usa la estructura condicional `if / elif / else` para comparar el número de estado introducido.

### B.2. Redes y Telemetría

31. **Calculadora de Hosts Útiles:** Solicita el número de bits dedicados a host en una red (por ejemplo, 8 bits) y calcula el número de direcciones de IP útiles disponibles ($2^n - 2$).

    💡 **Ayuda de Python:** Para elevar a una potencia en Python se utiliza el operador `**` (ejemplo: `2 ** bits - 2`).

32. **Conversor de Octeto a Binario:** Pida un número entero entre 0 y 255 (correspondiente a un octeto de una dirección IP) y muestre su representación en código binario.

    💡 **Ayuda de Python:** Utiliza la función integrada `bin(numero)`. Para quitar el prefijo `'0b'` de la salida puedes usar slicing: `bin(numero)[2:]`.

33. **Calculador de Variación de Latencia (Jitter):** Solicita al usuario la latencia máxima y la latencia mínima obtenidas en un test de red (en milisegundos) y calcula la diferencia entre ambas.

    💡 **Ayuda de Python:** Si pides una lista de valores numéricos, puedes usar las funciones `max()` y `min()` para obtener directamente los extremos.

### B.3. Monitorización de Sistemas

34. **Alertas de Uso de CPU:** Pida el porcentaje de uso de CPU actual y muestre una alerta: "OK" (si es menor o igual a 50%), "PRECAUCIÓN" (entre 51% y 85%) o "ALERTA

CRÍTICA" (mayor a 85%).

💡 **Ayuda de Python:** Utiliza condicionales anidados o encadenados mediante `if`, `elif` y `else` evaluando rangos numéricos.

35. **Proyección de Espacio en Disco:** Dado el espacio ocupado actual por los logs (en GB) y un crecimiento diario fijo, calcula cuánto espacio ocuparán dentro de 30 días.

    💡 **Ayuda de Python:** Aplica una operación matemática sencilla acumulando el crecimiento: `espacio_final = espacio_actual + (crecimiento_diario * 30)`.

36. **Generador de Nombres para Backup:** Solicita el nombre de un archivo (ej. `"datos.db"`) y añade como prefijo la palabra `"backup_"` y como sufijo la extensión `".bak"` para simular una rotación simple.

    💡 **Ayuda de Python:** Puedes concatenar cadenas directamente con el operador `+` o mediante f-strings: `f"backup_{nombre_archivo}.bak"`.

*(Nota: Estas actividades se corregirán gradualmente en las sesiones prácticas del aula para verificar la correcta asimilación e implementación de los algoritmos y del estilo de programación, algunas actividades formarán parte de la práctica correspondiente a la unidad).*